

Cross-Domain Evaluation of Retrieval-Augmented Generation (RAG) Systems

A Controlled Experimental Study of Domain Mismatch in RAG Pipelines

Aditya Jekkula · Ibaad Mohammed · Nate Dailey

Advanced Topics in Artificial Intelligence

May 2026

Abstract

Retrieval-Augmented Generation (RAG) systems combine information retrieval with large language model generation to produce grounded, factual answers. While RAG has shown strong performance in controlled settings, its behavior under domain mismatch — where the query originates from a different domain than the indexed corpus — remains poorly understood. This paper presents a controlled experimental study evaluating RAG performance across three datasets: SQuAD (structured Wikipedia QA), Natural Questions (open-domain Google search queries), and TriviaQA (trivia and quiz knowledge). We evaluate two retrieval methods (BM25 and dense retrieval using MiniLM + FAISS), a no-retrieval LLM baseline (Qwen 2.5-0.5B-Instruct), and their combinations across six cross-domain scenarios. Our results show that in-domain RAG significantly improves over the LLM-only baseline (SQuAD F1: 0.028 $\rightarrow$ 0.126), while cross-domain RAG shows near-zero, inconsistent, and sometimes negative gains. Failure analysis reveals that retrieval failure is the dominant failure mode in cross-domain settings (93–95% of cases), while generation failure dominates in challenging in-domain settings. These findings highlight the critical importance of corpus-query domain alignment in practical RAG deployments.

1. Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of natural language tasks. However, they suffer from well-documented limitations: they can hallucinate facts not present in their training data, their knowledge has a fixed cutoff date, and they cannot access proprietary or domain-specific information. Retrieval-Augmented Generation (RAG) addresses these limitations by augmenting the generation process with a retrieval step that fetches relevant context from an external knowledge corpus before generating a response.

Despite the growing adoption of RAG systems, a critical but underexplored question remains: how robust are these systems when the query domain does not match the domain of the indexed corpus? In real-world deployments, this mismatch is common. A system built on a medical literature corpus may receive general-knowledge questions; a customer support system may receive queries about topics not covered in its documentation. Understanding how domain mismatch propagates through the retrieval and generation pipeline is essential for building reliable RAG systems.

This work presents a controlled experimental study of RAG under domain mismatch. Unlike prior work that focuses on maximizing performance within a single domain, we deliberately construct cross-domain evaluation scenarios to study failure modes. Our contributions are:

- A systematic evaluation of BM25 and dense retrieval across six domain scenarios — three in-domain and three cross-domain.
- An independent evaluation of each pipeline component (retrieval and generation) to isolate failure modes.
- A failure taxonomy classifying failures as retrieval failure, generation failure, or success, with qualitative spot-check examples.
- Empirical evidence that cross-domain retrieval actively hurts generation quality below the no-retrieval baseline.

2. Related Work

2.1 Retrieval-Augmented Generation

RAG was formally introduced by Lewis et al. (2020) as a method combining a parametric memory (the LLM) with a non-parametric memory (a dense retrieval index). The original formulation used Dense Passage Retrieval (DPR) as the retriever and BART as the generator. Subsequent work has explored various retriever architectures, including sparse BM25 (Robertson & Zaragoza, 2009), dense bi-encoders (Karpukhin et al., 2020), and hybrid approaches that combine both. Izacard & Grave (2021) introduced Fusion-in-Decoder, demonstrating that retrieving multiple passages and fusing them during decoding improves performance.

2.2 Domain Mismatch in NLP

Domain mismatch has been studied extensively in the context of machine translation, sentiment analysis, and named entity recognition (Blitzer et al., 2006; Daume III, 2007). In the context of question answering, Talmor & Berant (2019) showed that QA models trained on one dataset do not transfer well to others, even when both are general-domain Wikipedia QA. For RAG

specifically, Shi et al. (2023) showed that irrelevant retrieved passages can confuse LLMs into producing worse answers than no retrieval at all — a finding our work empirically confirms across multiple domain pairs.

2.3 Evaluation of RAG Systems

Standard RAG evaluation uses Exact Match (EM) and token-level F1 scores adapted from the SQuAD benchmark (Rajpurkar et al., 2016). Recall@K and Mean Reciprocal Rank (MRR) are used to evaluate retrieval quality independently of generation. Recent work (Es et al., 2023; Gao et al., 2023) has proposed more comprehensive evaluation frameworks such as RAGAS, which evaluates faithfulness, answer relevancy, and context relevancy. Our study focuses on classical metrics while supplementing with qualitative analysis, as our datasets use short, extractive ground truth answers where token-level overlap provides a simple and interpretable measure of performance. Additionally, these metrics are computationally efficient to compute at scale, making them well suited to our experimental setup, while more complex frameworks (such as RAGAS) introduce additional model-based evaluation overhead that is beyond the scope of this study.

3. Methodology

3.1 Experimental Design

Our study is structured as a controlled experiment with three independent variables: (1) the query dataset, (2) the retrieval corpus, and (3) the retrieval method. All other components — the generator model, chunking strategy, prompt format, and evaluation metrics — are held constant across all experiments. This design ensures that observed performance differences can be attributed to domain mismatch and retrieval method rather than confounding factors.

We evaluate six scenarios by varying query and corpus domain:

Scenario	Query Domain	Corpus Domain	Type
A	SQuAD	SQuAD	In-domain
B	Natural Questions	Natural Questions	In-domain
C	TriviaQA	TriviaQA	In-domain
D	SQuAD	Natural Questions	Cross-domain
E	SQuAD	TriviaQA	Cross-domain
F	TriviaQA	SQuAD	Cross-domain

3.2 Datasets

Three datasets were selected to provide meaningful domain contrast while maintaining consistent answer format (short extractive spans) for fair evaluation across all scenarios.

3.2.1 SQuAD (Stanford Question Answering Dataset)

SQuAD (Rajpurkar et al., 2016) consists of questions posed over Wikipedia passages where answers are extractive spans guaranteed to appear in the passage. Questions are passage-specific — they are written with the source passage in front of the annotator, resulting in highly precise, context-dependent questions. We use 200 evaluation samples and a separate corpus pool of 5,000 passages.

Why SQuAD: SQuAD serves as our primary in-domain baseline. Its clean extractive structure and large corpus pool make it ideal for demonstrating RAG at its best. It is the dataset where retrieval is most beneficial.

Alternative considered: TriviaQA (web subset) was the original proposal. It was replaced because web-sourced TriviaQA contains noisy, inconsistently formatted passages that introduce confounding variables in the comparison. The rc (reading comprehension) subset used here is cleaner.

3.2.2 Natural Questions (NQ)

Natural Questions (Kwiatkowski et al., 2019) consists of real Google search queries with short extractive answers from Wikipedia. Unlike SQuAD, questions were not written with a passage in front of the annotator, resulting in more natural, open-ended queries. We use 200 evaluation samples and a corpus pool of 2,000 additional passages extracted from the streaming dataset.

Why NQ: NQ provides a middle-ground domain: same general knowledge base as SQuAD (Wikipedia) but structurally different query style (search queries vs. passage-specific questions). This allows us to study retrieval degradation due to query style mismatch even within the same knowledge domain.

Challenge encountered: NQ passages are delivered as raw HTML from Google's pipeline. Initial processing using a simple HTML tag stripper left CSS, JavaScript, and Wikipedia navigation boilerplate in the chunks (e.g., .mw-spinner CSS classes, List of proxy wars page navigation). This caused BM25 to retrieve irrelevant chunks based on CSS keyword overlap and made dense retrieval embeddings noisy. The fix required stripping script and style elements before tag removal, then collapsing all whitespace.

The NQ HTML cleaning pipeline:

```
context = re.sub(r'<(script|style)(^>)*>.*?</(script|style)>', '', html,
flags=re.DOTALL)
context = re.sub(r'<[^>]+>', ' ', context) # remove HTML tags
context = re.sub(r'\{[^}]+\}', ' ', context) # remove CSS blocks
context = re.sub(r'\s+', ' ', context).strip() # collapse whitespace
```

3.2.3 TriviaQA (rc subset)

TriviaQA (Joshi et al., 2017) consists of trivia questions sourced from quiz leagues, trivia websites, and books. Topics span entertainment, sports, history, science, and geography — domains genuinely distinct from Wikipedia-structured text. The rc (reading comprehension) subset pairs each question with relevant web documents containing the answer. We use 200 evaluation samples and a corpus pool of 3,000 passages.

Why TriviaQA instead of PubMedQA: The original proposal used PubMedQA as the third dataset. However, PubMedQA's ground truth answers are long-form author-written conclusions that do not appear verbatim in the source abstracts. This caused Recall@K to be 0% regardless

of retrieval quality, making quantitative evaluation impossible. TriviaQA provides short extractive answers directly comparable to SQuAD and NQ while offering stronger domain contrast.

Domain contrast with SQuAD/NQ: SQuAD and NQ are both Wikipedia-grounded. TriviaQA is sourced from trivia and entertainment databases. A SQuAD question like 'What is in front of the Notre Dame Main Building?' has no meaningful match in a trivia corpus about celebrity births, chart music, or Nobel Prize history.

3.3 Text Chunking

Each passage is split into overlapping sentence-aware chunks before indexing. Chunking improves retrieval precision by allowing the retriever to return the most relevant sub-passage rather than an entire document.

- Maximum chunk size: 250 words
- Overlap: 50 words between consecutive chunks to avoid losing context at boundaries
- Sentence-aware splitting using NLTK `sent_tokenize` (never cuts mid-sentence)
- Deduplication: identical chunks removed to prevent BM25 from returning the same chunk multiple times

Resulting corpus sizes:

Dataset	Corpus Pool	Unique Chunks
SQuAD	5,000 passages	842
Natural Questions	2,000 passages	1,983
TriviaQA	3,000 passages	5,409

The chunking function:

```
def chunk_text(text, max_words=250, overlap=50):
    sentences = sent_tokenize(text)
    chunks, current_chunk, current_len = [], [], 0
    for sentence in sentences:
        words = sentence.split()
        if current_len + len(words) > max_words and current_chunk:
            chunks.append(' '.join(current_chunk))
            overlap_words = ' '.join(current_chunk).split()[-overlap:]
            current_chunk = [' '.join(overlap_words)]
            current_len = len(overlap_words)
        current_chunk.append(sentence)
        current_len += len(words)
    if current_chunk:
        chunks.append(' '.join(current_chunk))
    return chunks
```

3.4 Retrieval Methods

3.4.1 BM25 (Sparse Retrieval)

BM25 (Best Match 25) is a classical probabilistic information retrieval function (Robertson & Zaragoza, 2009). It ranks documents by term frequency weighted by inverse document frequency and document length normalization. BM25 is purely lexical — it has no understanding of semantic meaning and relies entirely on keyword overlap between the query and the document.

We use the `rank_bm25` library (BM25Okapi implementation). A deduplication filter prevents the same chunk from appearing multiple times in the top-K results — an issue that arises when multiple very similar chunks receive identical BM25 scores.

```
def retrieve_bm25(query, bm25_index, chunks, k=5):
    scores = bm25_index.get_scores(query.split())
    sorted_indices = np.argsort(scores)[::-1]
    results, seen = [], set()
    for idx in sorted_indices:
        text = chunks[idx]
        if text not in seen:
            results.append((text, float(scores[idx])))
            seen.add(text)
        if len(results) == k: break
    return results
```

Why BM25: BM25 is a strong baseline that has been competitive with dense methods on many benchmarks. It is computationally cheap and interpretable — high scores directly indicate keyword overlap. Including BM25 allows us to assess whether semantic understanding (dense retrieval) provides a meaningful advantage, and whether that advantage holds under domain mismatch.

3.4.2 Dense Retrieval (FAISS + MiniLM)

Dense retrieval encodes both queries and chunks into fixed-length embedding vectors using a bi-encoder model. Retrieval is performed by nearest-neighbor search in the embedding space. We use `all-MiniLM-L6-v2` (Wang et al., 2020) as the embedding model — a distilled BERT model fine-tuned on 1 billion sentence pairs for semantic similarity tasks. Embeddings are indexed using `FAISS IndexFlatL2` (Johnson et al., 2019), which performs exact L2 distance search.

```
def retrieve_dense(query, faiss_index, chunks, k=5):
    q_emb = embed_model.encode([query], convert_to_numpy=True)
    D, I = faiss_index.search(q_emb, k * 3)
    results, seen = [], set()
    for dist, idx in zip(D[0], I[0]):
        if idx < len(chunks):
            text = chunks[idx]
            if text not in seen:
                results.append((text, float(dist)))
                seen.add(text)
```

```

        if len(results) == k: break
    return results

```

Why MiniLM over larger models: MiniLM-L6-v2 offers a strong accuracy-efficiency tradeoff for general-purpose semantic similarity. Larger models (e.g., MPNet, E5-large) would provide stronger in-domain retrieval but at significantly higher indexing and query cost. For a study focused on domain mismatch rather than peak retrieval performance, MiniLM is appropriate. Note that MiniLM was not fine-tuned on any of our three datasets, meaning its performance reflects general semantic similarity rather than domain-specific adaptation.

Why FAISS IndexFlatL2: Exact search is used rather than approximate nearest neighbor (ANN) methods like IVF or HNSW. Given our corpus sizes (under 6,000 chunks), exact search is fast enough and avoids the approximation errors introduced by ANN indexes, which could confound retrieval quality measurements.

3.5 Generation Model

We use Qwen 2.5-0.5B-Instruct (Alibaba Cloud, 2024) as the generator for all experiments. The model is loaded in float16 precision on an NVIDIA GTX 1650. A single generator is used across all experiments to isolate the effect of retrieval — any performance differences can be attributed to what the model received as context, not to differences in generation capability.

Two prompt templates are used:

RAG prompt: The model is instructed to answer using the provided context. The top `K_GENERATE = 3` retrieved chunks are concatenated and prepended to the question. We considered including a note in the prompt to “ONLY use information in the context,” but decided this was too strict. Instead, prompting the model to “use information in the context” is more representative of a real-world RAG system.

```

'Answer using the information in the context below.'
'Give a concise, direct answer.'
'Context: {context}'
'Question: {question}'
'Answer:'

```

LLM-only prompt: No context is provided. The model must answer from its parametric knowledge alone.

```

'Answer the following question as accurately and concisely as possible.'
'Question: {question}'
'Answer:'

```

Generation cleanup: An important engineering challenge was that the model occasionally continued generating beyond the answer — producing fake follow-up questions, injecting system prompt text, or repeating the question. This was fixed by splitting the output on multiple stop tokens:

```

answer = result.split('Answer:')[1].strip()
answer = answer.split('\n')[0].strip()
for stop in ['Human:', 'Question:', 'You are an AI', 'User will']:

```



```
answer = answer.split(stop)[0].strip()
```

Why Qwen 2.5-0.5B: Qwen 2.5-0.5B-Instruct was selected for its compact size and reasonable instruction-following capability. Larger models would increase runtime beyond available GPU memory for N=200 samples.

3.6 Evaluation Metrics

3.6.1 Retrieval Metrics

Recall@K: Binary measure of whether the correct answer appears within the top-K retrieved chunks. We use token overlap rather than exact substring matching — a chunk is considered a hit if 80% or more of the answer tokens appear in the chunk. This is more robust for short answers (e.g., '1983', 'Real Madrid') that may not survive exact substring matching in 250-word chunks.

```
def recall_at_k(answer, retrieved_chunks):
    ans_tokens = set(normalize(answer).split())
    for chunk in retrieved_chunks:
        chunk_tokens = set(normalize(chunk).split())
        overlap = ans_tokens & chunk_tokens
        if len(overlap) / len(ans_tokens) >= 0.8:
            return 1
    return 0
```

MRR (Mean Reciprocal Rank): Measures the rank of the first correct chunk. MRR=1.0 means the correct chunk was ranked first; MRR=0.5 means it was ranked second. MRR is more informative than Recall@K because it rewards retrievers that rank the correct chunk higher, not just those that include it somewhere in the top K.

3.6.2 Generation Metrics

Exact Match (EM): 1 if the normalized prediction exactly matches the normalized ground truth, 0 otherwise. Normalization lowercases, removes punctuation, and strips whitespace. EM is a strict metric — 'Saint Bernadette Soubirous.' and 'Saint Bernadette Soubirous' differ only by a period but EM=0.

F1 Score: Token-level overlap between prediction and ground truth, and our primary comparative metric across all conditions. F1 is computed as the harmonic mean of precision and recall, where precision measures the fraction of predicted tokens that appear in the ground truth (penalizing verbose or off-topic answers) and recall measures the fraction of ground truth tokens that appear in the prediction (penalizing incomplete answers). F1 is more forgiving than EM and rewards partial matches — a prediction of “Saint Bernadette” for the ground truth “Saint Bernadette Soubirous” scores partial credit rather than zero.

Limitations of F1: Token F1 has a well-known weakness: it treats all tokens equally, meaning common function words such as “the,” “of,” and “in” contribute to the score the same as content-bearing tokens. A prediction that shares only stopwords with the ground truth will still receive a nonzero F1 — though typically very small — potentially inflating poor answers. This makes F1 a noisy signal when comparing conditions with low overall performance, as is the case in most of our cross-domain scenarios. A semantically-aware metric such as BERTScore would be more robust: rather than counting exact token matches, BERTScore computes cosine similarity

between contextual embedding vectors produced by a pretrained language model, capturing meaning-level overlap even when surface wording differs. For example, a prediction of “the author Sinclair Lewis” and a ground truth of “Sinclair Lewis” would receive only partial credit under token F1 but score highly under BERTScore due to strong semantic similarity. Adopting BERTScore as a complementary evaluation metric is a concrete point of improvement for future iterations of this project.

Why these metrics: EM and F1 are the standard SQuAD benchmark metrics, making our results directly comparable to prior work. They are appropriate for short extractive answers. For longer generated answers, BERTScore would be more appropriate but was excluded to keep the evaluation framework computationally tractable.

3.6.3 Failure Classification

Each example in the dense RAG evaluation is classified into one of three categories:

- Retrieval Failure: Recall@K = 0 (correct chunk not retrieved)
- Generation Failure: Recall@K = 1 but F1 < 0.3 (chunk retrieved but answer wrong)
- Success: Recall@K = 1 and F1 ≥ 0.3

4. Results

4.1 Retrieval Performance

Table 2 shows Recall@5 and MRR for both retrievers across all six scenarios.

Scenario	BM25 Recall@5	Dense Recall@5	BM25 MRR	Dense MRR	Winner
A: SQuAD→SQuAD (in-domain)	0.750	0.925	0.584	0.792	Dense
B: NQ→NQ (in-domain)	0.015	0.015	0.005	0.008	Tie
C: Trivia→Trivia (in-domain)	0.255	0.325	0.172	0.246	Dense
D: SQuAD→NQ (cross-domain)	0.015	0.005	0.009	0.001	BM25
E: SQuAD→Trivia (cross-domain)	0.055	0.050	0.031	0.021	BM25
F: Trivia→SQuAD (cross-domain)	0.025	0.070	0.019	0.044	Dense

Dense retrieval outperforms BM25 on in-domain scenarios A and C, while performing approximately equally to BM25 for scenario B. This demonstrates that semantic understanding may provide a meaningful advantage when queries and corpus share the same domain. In Scenario A (SQuAD), dense retrieval achieves 0.925 Recall@5 versus BM25's 0.750 — a substantial gap attributable to the passage-specific nature of SQuAD questions, which require understanding the relationship between question phrasing and passage content rather than simple keyword overlap.

Under cross-domain conditions, neither retriever performs well. Recall@5 drops to 0.005–0.070 across all cross-domain scenarios. Notably, BM25 slightly outperforms dense retrieval on Scenarios D and E. This reversal is explained by the nature of SQuAD questions: they contain distinctive proper nouns (Notre Dame, Virgin Mary, Grotto) that keyword-match accidentally against NQ and TriviaQA chunks mentioning the same entities in unrelated contexts. Dense retrieval, which encodes the semantic meaning of the question (e.g., 'architectural feature of a building'), fails to match trivia-style chunks about entertainment or history. BM25's keyword advantage in Scenarios D and E should not be interpreted as genuine retrieval quality — the matched chunks do not contain the correct answers.

Scenario B (NQ in-domain) shows near-zero recall for both retrievers (0.015). This is a persistent issue attributable to residual HTML artifacts in NQ chunks. Despite cleaning script/style blocks and CSS, some Wikipedia navigation boilerplate remains. Recall improvement for NQ is an area for future work.

4.2 LLM-Only Baseline

Dataset	EM	F1
SQuAD	0.000	0.028
Natural Questions	0.000	0.040
TriviaQA	0.010	0.040

The LLM-only baseline establishes what Qwen 2.5-0.5B knows without retrieval. SQuAD F1 (0.028) is low because SQuAD questions are passage-specific — they reference a particular document the model has not seen. NQ and TriviaQA F1 (both 0.040) are similarly low, reflecting the limited parametric knowledge of this small 0.5B parameter model.

These baseline numbers are critical reference points for interpreting RAG performance. Any RAG result below these numbers indicates that retrieval is actively hurting the model.

4.3 Full RAG Performance

Scenario	Type	LLM-Only F1	BM25-RAG F1	Dense-RAG F1	Dense vs. LLM-Only
A: SQuAD→SQuAD	In-domain	0.028	0.118	0.126	+0.098 ↑
B: NQ→NQ	In-domain	0.040	0.037	0.041	+0.001 ↑
C: Trivia→Trivia	In-domain	0.040	0.066	0.060	+0.019 ↑
D: SQuAD→NQ	Cross-domain	0.028	0.036	0.030	+0.002 ↑
E: SQuAD→Trivia	Cross-domain	0.028	0.034	0.026	-0.002 ↓
F: Trivia→SQuAD	Cross-domain	0.040	0.032	0.032	-0.008 ↓

The most striking finding is in Scenario A: dense RAG achieves F1=0.126 versus the LLM-only baseline of 0.028 — a 4.5x improvement. This confirms that when retrieval is reliable and in-domain, RAG provides meaningful value.

In Scenario F (TriviaQA queries against SQuAD corpus), dense RAG F1 (0.032) falls slightly below LLM-only F1 of 0.040 — a small but meaningful degradation. This is not simply because retrieval fails to find the answer; it is because retrieved SQuAD chunks actively confuse the generator into producing answers grounded in the wrong domain context.

Scenario C (TriviaQA in-domain) shows improvement with both BM25-RAG (0.066 vs. 0.040) and Dense-RAG (0.060 vs. 0.040). However, the 67.5% retrieval failure rate even for dense retrieval means the generator frequently receives irrelevant chunks even when querying its own corpus.

4.4 Failure Analysis

Scenario	Retrieval Failure	Generation Failure	Success
A: SQuAD→SQuAD (in-domain)	7.5%	86.0%	6.5%
B: NQ→NQ (in-domain)	98.5%	1.5%	0.0%
C: Trivia→Trivia (in-domain)	67.5%	29.5%	3.0%
E: SQuAD→Trivia (cross-domain)	95.0%	5.0%	0.0%
F: Trivia→SQuAD (cross-domain)	93.0%	7.0%	0.0%

The failure analysis reveals a clear pattern: retrieval failure is the dominant failure mode in cross-domain settings (93–95%), while generation failure is more prominent in in-domain settings. In Scenario A, success is only 6.5% — retrieval recall is high (92.5%) but generation failure dominates at 86.0%, reflecting the 0.5B model’s difficulty extracting precise short answers. Success rates are low in part due to our conservative definition of success ($F1 \geq 0.3$). Many predictions contain partially correct answers but include additional tokens or verbosity that reduce F1 below this threshold, causing them to be classified as generation failures.

The generation failure rate in Scenario A (86%) indicates that even when the correct chunk is retrieved, the model rarely extracts the right answer. This occurs in two patterns: (1) the model generates a longer, more verbose answer that has low token overlap with the short ground truth, and (2) the model confidently produces a wrong answer rather than extracting from the context. Scenario D exhibits near-total retrieval failure (99.5%), with negligible generation failures and no successful cases, so it was excluded from the table. The near-total retrieval failure of Scenario D reinforces the conclusion that cross-domain retrieval failure is the dominant failure mode.

5. Qualitative Analysis

This section presents representative examples from each scenario category. Examples are organized by outcome type: success, generation failure, and retrieval failure. All examples are from the Dense RAG evaluation unless otherwise noted.

5.1 Successful RAG (In-Domain, Scenario A)

The clearest success cases are SQuAD in-domain, where both retrievers reliably find the correct passage and the generator extracts a precise answer.

Field	Content
Question	To whom did the Virgin Mary allegedly appear in 1858 in Lourdes France?
Ground Truth	Saint Bernadette Soubirous
Retrieved Chunk (Dense)	Architecturally, the school has a Catholic character. Atop the Main Building's gold dome is a golden statue of the Virgin Mary. Immediately in front of the Main Building... (answer appears later in chunk)
LLM-Only	The Virgin Mary allegedly appeared to a group of people in Lourdes, France in 1858. This event is known as the "Lourdes Miracle"... (F1=0.000)
Dense-RAG	The Virgin Mary allegedly appeared to Saint Bernadette Soubirous in 1858 in Lourdes, France. This event is described in detail in the text provided. (F1=0.222, EM=0)
BM25-RAG	The Virgin Mary allegedly appeared to Saint Bernadette Soubirous in 1858 in Lourdes, France. (F1=0.353, EM=0)

This example illustrates the core value proposition of RAG: the LLM-only answer fails entirely (F1=0.000). With the correct passage retrieved, both retrievers produce the correct name, though verbose phrasing prevents exact match (EM=0). Retrieval substantially improves generation even if not perfectly extractive.

5.2 Generation Failure Despite Correct Retrieval (In-Domain, Scenario A)

Field	Content
Question	The Basilica of the Sacred Heart at Notre Dame is beside to which structure?
Ground Truth	the Main Building
Retrieved Chunk (BM25)	Because of its Catholic identity, a number of religious buildings stand on campus... (contains the answer)
BM25-RAG	The Basilica of the Sacred Heart at Notre Dame is located next to the Old College building. The Old College building, which stands on the site of Fr.... (F1=0.074)

Field	Content
Dense-RAG	The Basilica of the Sacred Heart is situated next to the Basilica of the Sacred Heart. This can be inferred from the following sentence in the given text... (F1=0.065)

Both retrievers found the correct chunk but both models confidently hallucinated wrong answers — BM25-RAG invented 'the Old College building' and Dense-RAG produced a circular answer placing the Basilica next to itself. Neither hedged; both were confidently wrong. This illustrates that correct retrieval is necessary but not sufficient — the 0.5B model's generation quality remains a bottleneck.

5.3 Cross-Domain Retrieval Failure Leading to Hallucination (Scenario E: SQuAD→Trivia)

Field	Content
Question	What is in front of the Notre Dame Main Building?
Ground Truth	a copper statue of Christ
BM25 Retrieved (Wrong Domain)	Round One Jeopardy Template / The name of Dr. Seuss's egg-hatching elephant... (Trivia chunk about trivia games)
Dense Retrieved (Wrong Domain)	Consisting of 27 structures resembling petals of the lotus flower that open onto a central hall around 40m high... (Trivia chunk about architecture)
BM25-RAG	The main building of Notre Dame Cathedral is located in front of the Notre Dame Main Building. (F1=0.091 — circular wrong answer)
Dense-RAG	The Notre Dame Main Building is behind the Basilica of Saint Mary Major. However, I'm sorry, but I cannot provide a concise, direct answer... (F1=0.070)
LLM-Only	The Notre Dame Main Building, also known as the Notre Dame Cathedral or simply Notre Dame, is a massive Gothic cathedral... (F1=0.073)

BM25 retrieved an unrelated Jeopardy template; Dense retrieved a lotus-flower architecture description — both complete misses. BM25-RAG generates a circular wrong answer; Dense-RAG produces a confused answer and hedges. Both produce answers near or below the LLM-only baseline of F1=0.073. This exemplifies the 'context poisoning' failure mode identified by Shi et al. (2023).

5.4 Cross-Domain: Model Correctly Identifies Missing Context (Scenario F: Trivia→SQuAD)

Field	Content
Question	Which American-born Sinclair won the Nobel Prize for Literature in 1930?
Ground Truth	Sinclair Lewis

Field	Content
Dense Retrieved (SQuAD corpus)	The rise of Hitler and other dictators in the 1930s forced numerous Catholic intellectuals to flee Europe... (Notre Dame history chunk)
LLM-Only	Sinclair Lewis won the Nobel Prize for Literature in 1930... (F1=0.103)
BM25-RAG	There is no information provided about an American-born Sinclair... The context does not include this information. (F1=0.057)
Dense-RAG	The context provided does not contain any information about this topic. (F1=0.039)

In this case, the model correctly identifies that the retrieved context is irrelevant and refuses to answer. This is the desired behavior when context is wrong — refusing is better than hallucinating. However, it still scores $F1=0$ because the ground truth is 'Sinclair Lewis' and the model produced no answer. The LLM-only baseline scores 0.148 by drawing on parametric knowledge. This demonstrates a fundamental tension in RAG: the instruction to 'answer from context' prevents hallucination but also prevents the model from using what it legitimately knows when retrieval fails.

5.5 TriviaQA In-Domain: Partial Success with Correct Retrieval (Scenario C)

Field	Content
Question	Which American-born Sinclair won the Nobel Prize for Literature in 1930?
Ground Truth	Sinclair Lewis
Retrieved Chunk (Both BM25 and Dense)	An American Nobel Prize in Literature - Nov 05, 1930 - HISTORY.com An American Nobel Prize in Literature...
LLM-Only	Sinclair Lewis won the Nobel Prize for Literature in 1930... (F1=0.103)
BM25-RAG	Sinclair Lewis won the Nobel Prize in Literature in 1930. He was born in Sauk Center, Minnesota... (F1=0.089)
Dense-RAG	The American-born Sinclair Lewis won the Nobel Prize in Literature in 1930. According to the provided context... (F1=0.087)

Both retrievers found the correct HISTORY.com article. All three conditions produce the correct answer. However, BM25-RAG and Dense-RAG both include additional context in their responses which dilutes token overlap with the short ground truth 'Sinclair Lewis'. The LLM-only answer scores $F1=0.103$, slightly higher than RAG-based answers ($F1=0.089$ and $F1=0.087$ respectively) because the RAG responses include additional verbose context that dilutes token overlap.

6. Engineering Challenges and Debugging

6.1 BM25 Duplicate Retrieval

Early in the project, BM25 returned the same chunk three times in the top-K results. This occurred because multiple identical or near-identical passages existed in the corpus (SQuAD reuses the same Wikipedia passage for many questions). When chunks are identical, BM25 assigns them equal scores and the top-K selection returns duplicates, producing fake diversity.

Fix: A seen-set deduplication filter was added to both BM25 and dense retrieval functions, ensuring all top-K results are unique.

```
seen = set()
for idx in sorted_indices:
    text = chunks[idx]
    if text not in seen:
        results.append((text, float(scores[idx])))
        seen.add(text)
```

6.2 SQuAD Corpus Too Small

The first version of the notebook built the SQuAD corpus from the same 200 evaluation samples, yielding only 43 unique chunks. SQuAD reuses passages across many questions, so the retrieval task was trivially easy — 92% recall was not meaningful because there were only 43 possible chunks to search.

Fix: The corpus pool was expanded to 5,000 SQuAD passages (separate from the 200 evaluation questions), yielding 842 unique chunks. Evaluation samples and corpus samples are strictly separated.

6.3 NQ HTML Contamination

Natural Questions passages are delivered as raw Wikipedia HTML. A naive tag stripper removed the tags but left behind CSS class definitions (`.mw-spinner{...}`), JavaScript functions, and Wikipedia navigation boilerplate. BM25 retrieved chunks about 'List of proxy wars' for a query about The Walking Dead because both the query and the CSS-contaminated chunks shared common tokens from the Wikipedia navigation sidebar.

Fix: A multi-stage cleaning pipeline was implemented: (1) strip script/style elements before tag removal to eliminate JavaScript and CSS, (2) strip remaining HTML tags, (3) remove CSS block patterns, (4) collapse all whitespace.

6.4 Generation Leakage

Qwen 2.5-0.5B-Instruct occasionally continued generating beyond the answer, producing artifacts such as: (1) fake follow-up questions ('Human: What year was...'), (2) injected system prompt text ('You are an AI assistant'), (3) repetition of the question. These artifacts inflated response length and reduced F1 by adding irrelevant tokens.

Fix: The answer extraction function was updated to split on multiple stop patterns after extracting the text after 'Answer:'.

6.5 PubMedQA Evaluation Breakdown

The original project design used PubMedQA as the third dataset. PubMedQA's ground truth answers are long-form conclusions written by paper authors, synthesizing findings from the abstract. These conclusions do not appear verbatim in the source abstract text. As a result, Recall@K was 0% regardless of retrieval quality — the correct answer could never be in any retrieved chunk. F1 was also meaningless because the ground truth was paragraph-length scientific text while predictions were sentence-length.

Fix: PubMedQA was replaced with TriviaQA (rc subset), which provides short extractive answers directly comparable to SQuAD and NQ. PubMedQA remains an interesting future direction for evaluating RAG with long-form answers using BERTScore or ROUGE rather than token F1.

7. Discussion

7.1 When Does RAG Help?

Our results show that RAG provides significant value when three conditions are met: (1) the query and corpus share the same domain, (2) the retrieval corpus is large and diverse enough to contain the answer, and (3) the LLM does not already know the answer from pretraining. Scenario A satisfies all three conditions and shows a 4.5x F1 improvement. When any of these conditions fails, RAG provides diminishing returns.

An interesting finding is that TriviaQA in-domain (Scenario C) shows only modest RAG improvement ($0.040 \rightarrow 0.066$ for BM25) despite being an in-domain scenario. The failure analysis reveals the cause: 67.5% retrieval failure rate. TriviaQA's trivia-style questions ('In which decade did Billboard magazine first publish an American hit chart?') do not keyword-match well against news article and trivia website chunks, even from the same corpus. This suggests that within-domain retrieval quality varies significantly by question type.

7.2 When Does RAG Hurt?

The most significant finding is that cross-domain RAG consistently performs below the LLM-only baseline. This is not simply because retrieval fails to find the answer — it is because the retrieved irrelevant context actively corrupts generation. The model is instructed to answer from the provided context, so when it receives SQuAD chunks about Notre Dame for a trivia question about Sinclair Lewis, it either generates a wrong answer (context poisoning) or refuses to answer (context abstention). Both outcomes score lower than the LLM-only prediction.

This finding has practical implications: deploying a RAG system with a mismatched corpus is worse than deploying no RAG at all. Systems should include a domain classifier or confidence threshold that falls back to LLM-only generation when retrieved context is likely irrelevant.

7.3 Dense vs. BM25: When Does Semantic Understanding Help?

Dense retrieval outperforms BM25 on in-domain tasks (Scenarios A and C), confirming that semantic understanding adds value when corpus and query share vocabulary and structure. The gap is largest for SQuAD (Dense: 0.925, BM25: 0.750) where passage-specific questions benefit most from semantic matching. In Scenario C (Trivia $\rightarrow$ Trivia), BM25 slightly edges Dense (0.066 vs 0.060).

Under cross-domain conditions, BM25 occasionally outperforms dense retrieval through accidental keyword matches on named entities. This reversal should not be interpreted as BM25 being 'better' for cross-domain retrieval — both retrievers fail equally, but BM25's failures sometimes produce higher F1 due to fortuitous token overlap with the correct answer in an otherwise irrelevant chunk.

8. Future Work

8.1 Domain-Aware Retrieval

The most direct remedy for cross-domain degradation is to add a domain classifier that routes queries to the appropriate corpus before retrieval. A lightweight classifier trained on the three dataset distributions could significantly reduce context poisoning. Alternatively, confidence-

based fallback — where the retriever abstains if the top-K similarity scores are below a threshold — would allow the system to fall back to LLM-only generation when retrieval is likely to fail.

8.2 Fine-Tuned Dense Retrieval

MiniLM-L6-v2 was not fine-tuned on any of our three datasets. Domain-specific fine-tuning of the embedding model using contrastive learning (e.g., with DPR-style positive/negative pairs) could substantially improve in-domain recall for TriviaQA and NQ, where retrieval currently fails at high rates even in-domain.

8.3 Hybrid Retrieval

A hybrid BM25 + dense approach using reciprocal rank fusion (RRF) could combine the keyword precision of BM25 with the semantic understanding of dense retrieval. Given that BM25 and dense retrieval sometimes retrieve different relevant chunks (as seen in qualitative examples), fusion may improve overall recall without significant computational cost.

8.4 Long-Form Answer Evaluation

Token F1 is not suitable for datasets with long-form answers. Future work should evaluate RAG on datasets like PubMedQA or ELI5 using BERTScore or ROUGE-L, which better capture semantic similarity for paragraph-length answers. This would allow re-evaluation of PubMedQA, which was excluded from the current study due to evaluation metric incompatibility.

8.5 Larger and More Diverse Models

Qwen 2.5-0.5B-Instruct was used as a single fixed generator. Evaluating multiple generator models (e.g., Mistral-7B, Llama-3-8B) would reveal whether generation failures are model-specific or systematic. Smaller models (e.g., Qwen 0.5B) are less likely to have memorized dataset answers, which would make the LLM-only baseline weaker and RAG's contribution more visible. Larger models may be better at context abstention — correctly refusing to answer when retrieved context is irrelevant.

8.6 Adversarial Cross-Domain Testing

Future work could construct adversarial cross-domain scenarios where retrieved chunks are semantically similar but factually wrong — for example, retrieving a passage about Notre Dame Paris for a question about Notre Dame Indiana. This would more precisely measure the model's ability to detect and reject misleading context.

8.7 More Robust Evaluation Metrics

While token-level F1 serves as a simple and widely used evaluation metric for extractive QA, it has notable limitations. Because F1 treats all tokens equally, it can assign nonzero scores to predictions that share only common function words with the ground truth, introducing noise—particularly in low-performance cross-domain settings. Additionally, F1 is sensitive to verbosity and does not capture semantic similarity beyond exact token overlap. Future work could incorporate a semantically-aware metric such as BERTScore, which compares contextual embeddings to better reflect meaning-level alignment between predictions and ground truth. Although excluded here due to computational cost, incorporating BERTScore alongside EM and F1 would provide a more robust evaluation framework for generated answers.

9. Conclusion

This paper presented a controlled experimental study of RAG systems under domain mismatch. Evaluating BM25, dense retrieval, and LLM-only baselines across six domain scenarios on 200 samples per dataset, we found:

- In-domain RAG with dense retrieval provides meaningful improvements over the LLM-only baseline (SQuAD F1: 0.028 $\rightarrow$ 0.126, a 4.5x improvement).
- Cross-domain RAG shows near-zero impact relative to the no-retrieval baseline, with small F1 deltas ranging from +0.002 to -0.008.
- Retrieval failure is the dominant failure mode in cross-domain settings (93–95%), while generation failure dominates in in-domain settings (86% for SQuAD despite 92.5% retrieval recall).
- Dense retrieval outperforms BM25 in-domain but shows no consistent advantage cross-domain. BM25 occasionally outperforms dense retrieval through accidental named-entity keyword matches.
- Context poisoning — where irrelevant retrieved chunks confuse the generator into producing worse answers — is a systematic failure mode that argues for confidence-based retrieval fallback in practical deployments.

These findings underscore that corpus-query domain alignment is not just a performance consideration but a correctness consideration: a RAG system deployed with a mismatched corpus can produce more confident wrong answers than a system with no retrieval at all. As RAG systems are increasingly deployed in production, domain-aware retrieval and fallback mechanisms are essential components of reliable system design.

References

- Blitzer, J., McDonald, R., & Pereira, F. (2006). Domain adaptation with structural correspondence learning. *Proceedings of EMNLP*.
- Daumé III, H. (2007). Frustratingly easy domain adaptation. *Proceedings of ACL*.
- Es, S., James, J., Anke, L. E., & Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. *arXiv:2309.15217*.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv:2312.10997*.
- Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. *Proceedings of EACL*.
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*.
- Joshi, M., Choi, E., Weld, D. S., & Zettlemoyer, L. (2017). TriviaQA: A reading comprehension dataset over trivia questions. *Proceedings of ACL*.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. T. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of EMNLP*.
- Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., ... & Petrov, S. (2019). Natural questions: A benchmark for question answering research. *Transactions of ACL*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Proceedings of NeurIPS*.
- Qwen Team, Alibaba Cloud. (2024). Qwen2.5 Technical Report. *arXiv:2412.15115*.
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. *Proceedings of EMNLP*.
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*.
- Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E., ... & Zhou, D. (2023). Large language models can be easily distracted by irrelevant context. *Proceedings of ICML*.
- Talmor, A., & Berant, J. (2019). MultiQA: An empirical investigation of generalization and transfer in reading comprehension. *Proceedings of ACL*.
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Proceedings of NeurIPS*.